

6. JavaScript moderno

6.1. Modo estricto

El modo estricto es una característica que aparece en el estándar EcmaScript5, aparecido en 2011. Su principal función es la de hacer que determinadas acciones que provocaban **errores silenciosos** o **efectos no deseados** se conviertan en **errores explícitos** a la hora de ejecutar los programas.

El modo estricto se puede activar:

- A nivel de *script*, indicándolo al principio del archivo.
- A nivel de *función*, indicándolo en la primera línea del cuerpo de la función.
- En los archivos de *módulo*, donde está **activado por defecto**.

! IMPORTANTE

No es buena idea mezclar código en modo normal y modo estricto. Por ello, es más recomendable utilizar el modo estricto a nivel de *script*.

Para activar el modo estricto, hay que escribir el texto `"use strict";` o `'use strict';` en la **primera línea** del archivo de *script* o función.

! IMPORTANTE

El modo estricto está **activado por defecto** en los **módulos**, por lo que no hay que hacer nada para activarlo.

Una de las ventajas más interesantes del modo estricto es que previene la creación de variables accidentales en el objeto global. Consideremos un programa en el que cometemos un error al escribir un nombre de variable:

```
let miVariable = 1;

// ...Código del programa...

// Nombre de variable mal escrito (la 'v' es minúscula)
mivariable = 25; // No se genera error; además, se crea una variable nueva
```

En modo normal JavaScript **no genera ningún error**, sino que directamente crea una nueva variable con el nombre `mivariable` en el objeto global. Ello puede hacer que el código falle y que sea muy difícil de depurar, ya que no hay ningún error que nos dé ninguna pista. Si utilizamos modo estricto, sí que se genera error:

```
"use strict";

let miVariable = 1;

// ...Código del programa...

// Nombre de variable mal escrito
mivariable = 25; // ReferenceError: mivariable is not defined
```

Este tipo de errores suele ser muy habitual al omitir la referencia a **this** en funciones que se utilizan como constructoras de objetos. Por ejemplo, consideremos la siguiente función constructora:

```
function Coche(a, b) {  
    this.marca = a;  
    this.modelo = b;  
}  
  
let c1 = new Coche("Seat", "Ibiza");  
  
console.log(c1.marca); // Seat  
console.log(c1.modelo); // Ibiza
```

Si se omite el uso de **this**, quedaría de la siguiente manera:

```
function Coche(a, b) {  
    marca = a; // Se crea una variable global con nombre 'marca'  
    modelo = b; // Se crea una variable global con nombre 'modelo'  
}  
  
let c1 = new Coche("Seat", "Ibiza"); // No se genera ningún error  
  
// Sin embargo...  
console.log(c1.marca); // undefined  
console.log(c1.modelo); // undefined  
  
// Y además se crean dos variables globales  
console.log(marca); // Seat  
console.log(modelo); // Ibiza
```

Existen muchos otros casos en los que JavaScript no genera errores explícitos. Por ello, el uso del modo estricto es recomendable, sobre todo, para programadores con poca experiencia con el lenguaje.

6.2. ¿Clases u objetos?

Tal como hemos visto en el apartado correspondiente, JavaScript ofrece diferentes maneras de trabajar con objetos, entre las que podemos destacar el uso de **funciones constructoras**, objetos con **Object.create** o las **clases**. La elección de una opción u otra dependerá de nuestras preferencias en el caso de crear programas nuevos o de la técnica utilizada en el caso de modificar programas existentes. Lo que sí es recomendable es **mantener la coherencia** y utilizar la misma técnica en todo el código de la aplicación.

Dicho esto, en ocasiones nos encontraremos con que algunas API o librerías nos *fuerzan* a utilizar un método concreto: es el caso de la tecnología **Web Components**, un estándar en desarrollo para crear componentes HTML que puedan ser reutilizados en las páginas. Esta API del navegador utiliza la sintaxis de **clases** en su documentación. A continuación, puedes ver un ejemplo:

```
customElements.define('mi-componente',  
    class extends HTMLElement {  
        constructor() {  
            super();  
            // ...  
        }  
    }  
);
```

6.3. Manejo de errores

El manejo de errores en JavaScript se realiza con los bloques `try...catch`. Estos bloques funcionan de manera parecida a otros lenguajes. Los errores que se capturan son los que aparecen **en tiempo de ejecución**.

La función `catch` define un parámetro, que es el error capturado. Dicho parámetro hace referencia a un objeto **Error**, que tiene **tres propiedades**:

- **name**. El nombre del error.
- **message**. El mensaje de error.
- **stack**. Información sobre la ruta de ejecución que ha provocado el error.

A continuación, se muestra un ejemplo:

```
try {  
    // Acceso a una propiedad de una variable que no existe  
    console.log(hola.noexiste);  
} catch(error) { // El parámetro puede tener cualquier nombre, no tiene por  
    // qué ser 'error'  
    console.log(error.name); // ReferenceError  
    console.log(error.message); // hola is not defined  
}
```

También es posible generar errores personalizados mediante `throw`:

```
try {  
    let a = "valornovalido";  
  
    if (a == "valornovalido") {  
        // Generamos un error con un mensaje personalizado  
        throw new Error("a no puede tener ese valor");  
    }  
} catch(err) {  
    console.log(err.name); // Error  
    console.log(err.message); // a no puede tener ese valor  
}
```

! IMPORTANTE

`try...catch` funciona de manera síncrona. Por tanto, no funciona correctamente con funciones asíncronas [que veremos en otra unidad posterior].

En ocasiones, puede ser interesante ejecutar un trozo de código **independientemente de que se haya generado o no un error**. Consideremos, por ejemplo, una petición a un recurso remoto cuya conexión hay que cerrar, tanto si se realiza con éxito como si no. En ese caso podemos utilizar el bloque **finally**:

```
try {  
    // Apertura de conexión  
    // Procesamiento de datos  
}  
} catch(err) {  
    // Captura de errores  
}  
} finally {  
    // Cierre de conexión  
    // Este bloque se ejecuta siempre, tanto si hay error como si no  
}
```

6.4. Evolución y futuro

JavaScript es un lenguaje en continua evolución. Goza de una enorme popularidad y además está estrechamente ligado a la web, hecho que potencia también su desarrollo. Algunas de las muchas características que se han añadido en los últimos años son:

- Operador *rest* y sintaxis *spread*.
- Promesas y funciones asíncronas.
- Generadores.
- *Proxies*.
- El sistema de módulos.
- Tipo de datos **BigInt**.
- Nuevas estructuras y sintaxis.
- Mejoras de sintaxis para la simplificación de estructuras de código.

Algunas de ellas las hemos estudiado en esta unidad. Otras las veremos en unidades posteriores (como las promesas y las funciones asíncronas), mientras que el resto (generadores, *proxies*) no las estudiaremos aquí por motivos de espacio y complejidad.

En paralelo a la evolución del lenguaje, se produce también una enorme evolución en las **API de los navegadores**, que continuamente están desarrollándose y ampliando funcionalidades. En la siguiente unidad estudiaremos algunas de ellas y veremos cómo se benefician también de la evolución del lenguaje.

Por ambos motivos, podemos esperar que JavaScript seguirá evolucionando en los próximos años, incorporando nuevas características que permitirán a los desarrolladores crear aplicaciones cada vez más complejas y potentes. Como futuro desarrollador, tienes el reto de seguir formándote a lo largo de tu carrera profesional para actualizar tus conocimientos y mantenerte al día.